

Bộ Câu Hỏi Phỏng Vấn NestJS (Intern / Fresher)

Câu 1: NestJS là gì?

Trả lời: NestJS là một framework để xây dựng ứng dụng backend bằng Node.js và TypeScript. Nó dựa trên kiến trúc module và sử dụng nhiều khái niệm giống Angular như decorators, DI (Dependency Injection), giúp việc phát triển backend trở nên có tổ chức và dễ mở rộng.

Câu 2: NestJS có ưu điểm gì so với Express?

Trả lời: - NestJS có kiến trúc rõ ràng (theo module)

- Hỗ trợ TypeScript đầy đủ
- Có Dependency Injection
- Dễ test, dễ bảo trì
- Tích hợp tốt với các công cụ như Swagger, TypeORM, GraphQL

Câu 3: Module trong NestJS là gì?

Trả lời: Module là nơi nhóm các thành phần liên quan (controller, service, provider) lại với nhau. Mỗi ứng dụng NestJS bắt đầu từ AppModule.

Câu 4: Controller là gì?

Trả lời: Controller xử lý các HTTP request từ client và trả lại response. Dùng các decorator như @Get(), @Post(), @Put(), @Delete() để định nghĩa route.

Câu 5: Service là gì?

Trả lời: Service chứa logic nghiệp vụ và thường được inject vào controller. Được đánh dấu bằng @Injectable().

Câu 6: Decorator là gì?

Trả lời: Decorator là một hàm đặc biệt gắn vào class/method/parameter để thêm metadata hoặc thay đổi hành vi.

Bộ Câu Hỏi Phỏng Vấn NestJS (Intern / Fresher)

Câu 7: DTO là gì? Tại sao cần dùng?

Trả lời: DTO (Data Transfer Object) là class định nghĩa shape của dữ liệu được gửi từ client. Giúp kiểm soát và validate dữ liệu đầu vào.

Câu 8: NestJS hỗ trợ validate dữ liệu như thế nào?

Trả lời: Thông qua thư viện class-validator và class-transformer. Dùng cùng với DTO để validate dữ liệu đầu vào.

Câu 9: Pipe là gì?

Trả lời: Pipe là một lớp để xử lý dữ liệu đầu vào, dùng để validate hoặc transform dữ liệu. Ví dụ: ValidationPipe, ParseIntPipe.

Câu 10: Middleware trong NestJS là gì?

Trả lời: Middleware là một hàm được gọi trước khi request đến controller. Thường dùng để xử lý logging, xác thực,...

Câu 11: Guard là gì?

Trả lời: Guard dùng để kiểm tra quyền truy cập vào route. Trả về true (cho phép) hoặc false (từ chối).

Câu 12: Interceptor là gì?

Trả lời: Interceptor can thiệp vào quá trình xử lý request/response. Có thể dùng để logging, mapping response, xử lý exception,...

Câu 13: Làm sao để xử lý lỗi trong NestJS?

Trả lời: NestJS có sẵn các exception như BadRequestException, NotFoundException,... hoặc có thể tạo custom Exception Filter.

Câu 14: NestJS có hỗ trợ ORM không?

Trả lời: Có. NestJS hỗ trợ TypeORM, Sequelize, Prisma,... Dễ dàng tích hợp với module

Bộ Câu Hỏi Phỏng Vấn NestJS (Intern / Fresher)

database.

Câu 15: Nest CLI dùng để làm gì?

Trả lời: Nest CLI giúp tạo module, controller, service nhanh chóng qua dòng lệnh. Ví dụ:
`nest g controller cats`

Câu 16: NestJS có thể dùng với Fastify không?

Trả lời: Có. Dù mặc định dùng Express, NestJS có thể dùng Fastify để tăng hiệu năng khi cấu hình lại Adapter.

Câu 17: Làm sao để viết RESTful API trong NestJS?

Trả lời: Tạo controller, định nghĩa các route với `@Get()`, `@Post()`,... và xử lý logic trong service. Có thể dùng Swagger để generate docs.

Câu 18: NestJS có hỗ trợ Swagger không?

Trả lời: Có. NestJS tích hợp rất tốt với Swagger để tạo tài liệu API tự động từ decorator.

Câu 19: Làm sao để dùng JWT Auth trong NestJS?

Trả lời: Dùng Passport + JWT strategy, tạo guard để xác thực token, và sử dụng `@UseGuards(AuthGuard('jwt'))` trong controller.